

MAINCRAFTS TECHNOLOGY

Internship Programme

Introduction to Embedded Systems & IoT Device Design

Task 1 Project Report

Smart Temperature Monitoring and Alert System

Intern Name	Pratibha Negali
Intern ID	MT8899
Domain	Embedded Systems & Internet of Things (IoT)
Organisation	Maincrafts Technology
Internship Duration	01 October 2026 to 15 October 2026 (2 Weeks)
Task	Embedded Systems Task 1
Project Option Chosen	Smart Temperature Monitor
Simulation Platform	Tinkercad Circuits

Table of Contents

1. Introduction to Embedded Systems
2. Component Study
3. Mini Project: Smart Temperature Monitor
4. Working Principle
5. Arduino Code
6. Code Explanation
7. Simulation Results
8. Real-World Applications
9. Future Enhancements
10. Conclusion

1. Introduction to Embedded Systems

1.1 What is an Embedded System?

An embedded system is a computer-based system designed to perform a specific function or set of functions within a larger electronic or mechanical system.

Unlike general-purpose computers, embedded systems are developed for dedicated applications. They typically consist of a controller, sensors, memory, and input/output interfaces.

In this project, the Arduino UNO acts as the controller, the TMP36 measures temperature, and the LED and buzzer provide alerts.

1.2 Applications of Embedded Systems

Embedded systems are commonly used in the following areas:

- Smart home systems
- Automotive systems
- Healthcare devices
- Robotics
- Agricultural automation
- Industrial automation
- Consumer electronics

1.3 Microcontroller vs Microprocessor

Feature	Microcontroller	Microprocessor
Function	Designed mainly for dedicated control applications	Used mainly for general-purpose computing
Memory	Usually integrated	Generally requires external memory
Peripherals	Usually integrated	Often externally connected
Power	Generally lower	Generally higher
Applications	Embedded systems and automation	Computers and high-performance systems

2. Component Study

2.1 Arduino UNO

Arduino UNO is a microcontroller-based development board used for embedded-system prototyping and automation. In this project, it:

- Reads the TMP36 sensor.

- Processes the temperature value.
- Compares it with the threshold.
- Controls the LED and buzzer.

2.2 ESP32

ESP32 is a microcontroller platform commonly used for IoT applications and wireless sensor systems. It can be used for:

- IoT monitoring
- Smart-home applications
- Wireless communication
- Automation

2.3 Raspberry Pi Pico

Raspberry Pi Pico is a compact microcontroller board suitable for embedded-system projects. Its applications include:

- Sensor-based systems
- Robotics
- Automation
- Embedded prototypes

2.4 Temperature Sensor

A temperature sensor measures the temperature of its surroundings. An example is the TMP36, which is used in this project to obtain the temperature reading.

2.5 Motion Sensor

A motion sensor detects movement in its surroundings. An example is the PIR sensor. Applications include security systems, automatic lighting, and motion detection.

2.6 Light Sensor

A light sensor detects the intensity of light. An example is the LDR (Light Dependent Resistor). Applications include automatic lighting systems and light-level monitoring.

3. Mini Project: Smart Temperature Monitor

3.1 Objective

The objective of this project is to design and simulate a Smart Temperature Monitoring and Alert System using an Arduino UNO and a TMP36 temperature sensor.

The system continuously monitors temperature and activates an LED and a buzzer when the temperature reaches or exceeds a predefined threshold of 30°C. This corresponds to the Smart Temperature Monitor option specified in the internship task.

3.2 Components Used

Component	Purpose
Arduino UNO	Main controller
TMP36	Temperature sensing
LED	Visual alert
Piezo buzzer	Audible alert
220 Ω resistor	LED current limiting
Breadboard	Circuit assembly
Jumper wires	Electrical connections

3.3 Circuit Connections

Component	Connection
TMP36 Left Pin	5V
TMP36 Middle Pin	A0
TMP36 Right Pin	GND
LED (anode)	Digital pin D8 through a 220 Ω resistor
LED (other terminal)	GND
Piezo buzzer (+)	Digital pin D9
Piezo buzzer (-)	GND

3.4 Circuit Diagram

The circuit was assembled and simulated in Tinkercad Circuits, as shown in Figure 1.

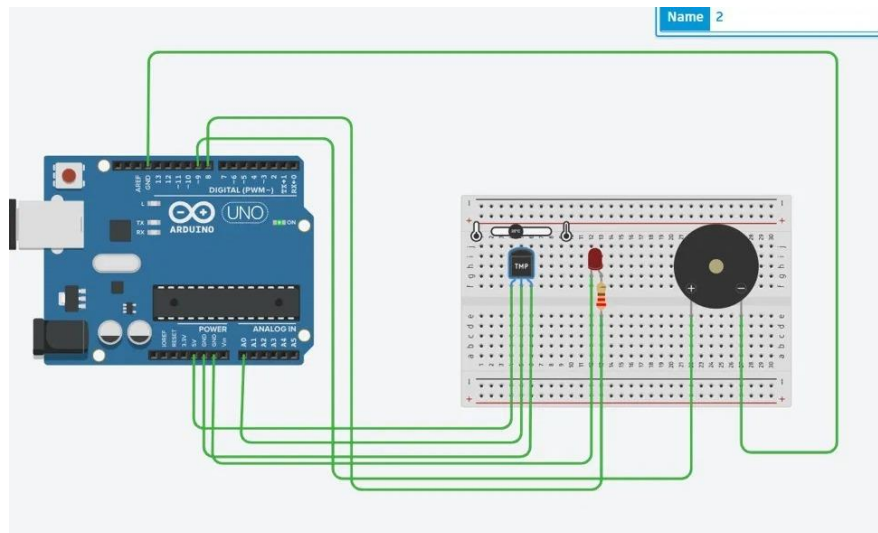


Figure 1: Smart Temperature Monitoring System simulated using Tinkercad

4. Working Principle

The system operates according to the following sequence:

Temperature → TMP36 → Arduino UNO → Threshold Comparison → Alert

1. The TMP36 senses the surrounding temperature.
2. It produces an analog voltage corresponding to the temperature.
3. The Arduino reads this signal through analog pin A0.
4. The analog reading is converted into temperature in degrees Celsius.
5. The Arduino compares the measured temperature with the 30°C threshold.
6. If the temperature is below 30°C, the LED and buzzer remain OFF.
7. If the temperature is 30°C or above, the LED and buzzer turn ON.
8. The temperature value is displayed through the Serial Monitor.

5. Arduino Code

The complete source code used for the project is given below.

```
// Smart Temperature Monitoring and Alert System
// Board: Arduino UNO | Sensor: TMP36 | Alerts: LED (D8) and piezo buzzer (D9)

const int tempPin = A0;      // TMP36 output connected to analog pin A0
const int ledPin = 8;        // LED connected to digital pin 8
const int buzzerPin = 9;     // Piezo buzzer connected to digital pin 9

const float threshold = 30.0; // Alert threshold in degrees Celsius

void setup() {
  Serial.begin(9600);        // Start serial communication at 9600 baud
```

```
pinMode(ledPin, OUTPUT);    // Configure LED pin as output
pinMode(buzzerPin, OUTPUT); // Configure buzzer pin as output
}

void loop() {

    // Read the raw analog value (0 to 1023) from the TMP36
    int sensorValue = analogRead(tempPin);

    // Convert the raw value to voltage (0 to 5 V)
    float voltage = sensorValue * (5.0 / 1023.0);

    // Convert voltage to temperature in Celsius
    // TMP36: 0.5 V offset at 0 degrees C, 10 mV per degree C
    float temperature = (voltage - 0.5) * 100.0;

    // Display the temperature on the Serial Monitor
    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" C");

    // Compare with the threshold and control the alerts
    if (temperature >= threshold) {
        digitalWrite(ledPin, HIGH);    // LED ON
        digitalWrite(buzzerPin, HIGH); // Buzzer ON
    }
    else {
        digitalWrite(ledPin, LOW);      // LED OFF
        digitalWrite(buzzerPin, LOW);   // Buzzer OFF
    }

    delay(1000);                      // Wait 1 second before the next reading
}
```

6. Code Explanation

6.1 Pin Declaration

```
const int tempPin = A0;
const int ledPin = 8;
const int buzzerPin = 9;
```

These statements define the Arduino pins connected to the temperature sensor, LED, and buzzer.

6.2 Threshold

```
const float threshold = 30.0;
```

The alert threshold is set to 30°C.

6.3 Sensor Reading

```
int sensorValue = analogRead(tempPin);
```

The Arduino reads the analog output of the TMP36 through pin A0. The value ranges from 0 to 1023.

6.4 Temperature Conversion

```
float voltage = sensorValue * (5.0 / 1023.0);  
float temperature = (voltage - 0.5) * 100.0;
```

The analog reading is first converted into voltage and then into temperature in degrees Celsius. The TMP36 outputs 0.5 V at 0°C and changes by 10 mV per degree Celsius, which is why the offset of 0.5 V is subtracted and the result is multiplied by 100.

6.5 Alert Condition

```
if (temperature >= threshold)
```

The Arduino checks whether the temperature has reached or exceeded 30°C. If the condition is true, both the LED and the buzzer are activated. Otherwise, both remain OFF.

7. Simulation Results

The project was successfully simulated using Tinkercad Circuits. Two test cases were carried out, one below the threshold and one above it.

7.1 Test Case 1: Normal Temperature

Condition: Temperature below 30°C. The Serial Monitor displayed a steady reading of 24.78°C.

Output:

- LED: OFF
- Buzzer: OFF
- Temperature displayed in the Serial Monitor

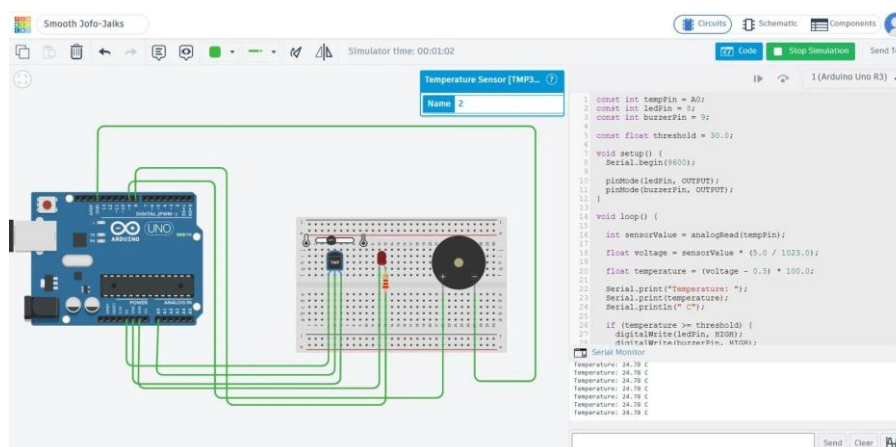


Figure 2: Normal temperature condition (24.78°C, LED and buzzer OFF)

7.2 Test Case 2: High Temperature

Condition: Temperature above 30°C. The sensor was set to a higher value, and the Serial Monitor displayed a reading of 46.77°C.

Output:

- LED: ON
- Buzzer: ON
- Temperature displayed in the Serial Monitor

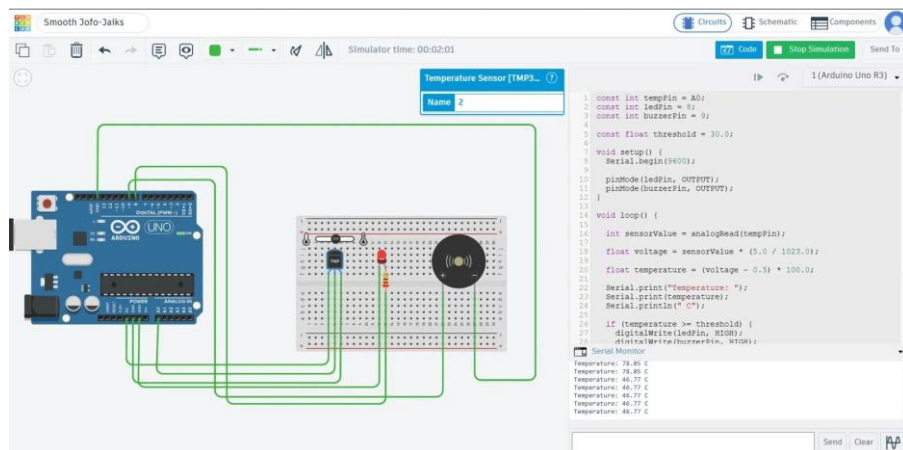


Figure 3: High-temperature alert condition (46.77°C, LED and buzzer ON)

7.3 Result Summary

Test Case	Temperature	Condition	LED	Buzzer
Normal temperature	24.78°C	Below 30°C	OFF	OFF
High temperature	46.77°C	Above 30°C	ON	ON

The simulation successfully demonstrated that the system activates the visual and audible alerts when the temperature crosses the predefined threshold.

8. Real-World Applications

The system can be adapted for the following applications:

- **Smart home:** temperature monitoring and overheating alerts.
- **Industrial:** monitoring the temperature of machines and equipment.
- **Agriculture and environment:** monitoring greenhouse and storage conditions.
- **Healthcare:** maintaining controlled environments such as medicine storage.
- **Electronics:** detecting overheating in electronic equipment.

9. Future Enhancements

The prototype can be enhanced by using an ESP32 and adding Internet connectivity. Possible improvements include:

- Cloud-based temperature monitoring
- Mobile notifications
- Real-time temperature graphs
- Data logging
- Remote monitoring
- Adjustable temperature thresholds

A future version could follow this architecture:

Temperature Sensor → ESP32 → Wi-Fi → Cloud → Mobile Dashboard

10. Conclusion

The Smart Temperature Monitoring and Alert System was successfully designed and simulated using an Arduino UNO and a TMP36 temperature sensor in Tinkercad. The system continuously monitors temperature and compares the measured value with a predefined threshold of 30°C. When the temperature reaches or exceeds the threshold, the LED and buzzer are activated to provide an immediate warning.

The project provided practical exposure to embedded systems, sensors, analog signal processing, microcontroller programming, and basic IoT-oriented system design.